

网络安全技术课程实验报告

实验二



实验名称：网络嗅探与数据包伪造_____

姓名：____段俊宇_____

学号：____2313815_____

专业：____信息安全、法学双学位班____

提交日期：2026 年 4 月 11 日_____

一、实验目的

1. 了解数据包嗅探和伪造的工作原理。
2. 理解计算机网络中的主被动攻击安全风险。
3. 了解 Scapy 库的使用方法。

二、实验要求及要点

结合 SEED Lab 教程，实现嗅探与数据包伪造实验。直观理解计算机网络中的主被动攻击安全风险，并深入理解计算机网络工作原理。

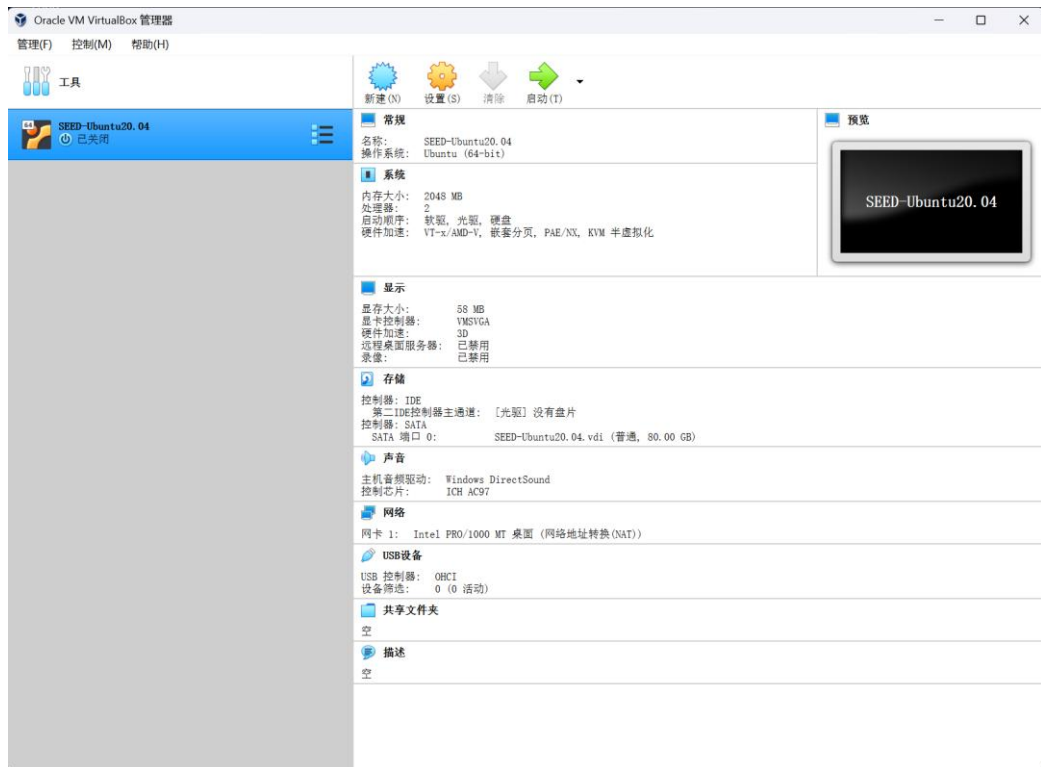
注意：只需要选取其中一个 Lab Task Set 即可。使用 Python 的 scapy 库比较简单，因此推荐选择 Task Set 1。

三、实验内容

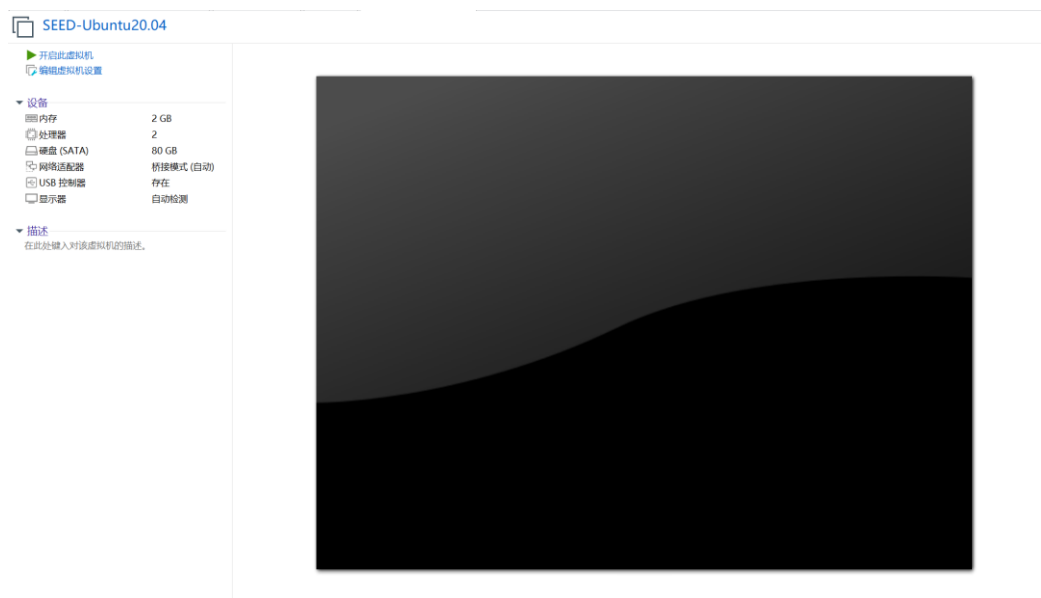
3.1 环境配置

在实验过程中我按照 SEED Lab 官网给出的虚拟机配置方法，下载了 VirtualBox 和 SEED-Ubuntu20.04 虚拟机，该虚拟机已经配置好了全部的环境，因此便于实验。在下载 VirtualBox 时需要注意版本为 6.1.16，这样才能导入官网的.vdi 文件。

如果提示 6.1.16 版本的安装程序无法使用，则需要在 Windows 安全中心-设备安全性中关闭内核完整性并重启电脑，这样就可以正常安装了。之后按照 github 教程配置虚拟机即可，安装完成后如下所示：



在 VirtualBox 中运行该虚拟机发现无法打开，因此我决定将其转换为 VMware 的虚拟机进行实验。



接下来将 Labsetup 放入虚拟机中并解压，在使用 docker-compose up 命令时发现无法连接 docker 并启动容器，换源也不行，因此采用端口转发的方式，使用本机的代理访问 docker。

首先在 VMware 的虚拟网络编辑器中选择更改设置，在 NAT 设置中添加端口转发，我设置的本机端口为 2222，虚拟机端口为 22，IP 是 SEED-

Ubuntu20.04 的 IP 地址，类型为 TCP，如下所示：

NAT 设置

网络: vmnet8
子网 IP: 192.168.66.0
子网掩码: 255.255.255.0
网关 IP(G): 192.168.66.2

端口转发(F)

主机端口	类型	虚拟机 IP 地址	描述
2222	TCP	192.168.66.137: 22	

添加(A)... 移除(R) 属性(P)

高级

☒ 允许活动的 FTP(T)
☒ 允许任何组织唯一标识符(O)
UDP 超时(以秒为单位)(U): 30
配置端口(C): 0
☐ 启用 IPv6(E)
IPv6 前缀(6): fd15:4ba5:5a2b:1008::/64

DNS 设置(D)... NetBIOS 设置(N)...

确定 取消 帮助

接下来在本机打开命令行，使用 `ssh -p 2222 seed@127.0.0.1` 命令连接虚拟机，密码即为 seed 账户密码。这样就完成了虚拟机 SSH 连接，在本机的命令行使用虚拟机终端。

```
seed@VM: ~  
Microsoft Windows [版本 10.0.26200.8037]  
(c) Microsoft Corporation。保留所有权利。  
  
C:\Users\LIANXIANG>ssh -p 2222 seed@127.0.0.1  
  
C:\Users\LIANXIANG>ssh -p 2222 seed@127.0.0.1  
seed@127.0.0.1's password:  
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/advantage  
  
0 updates can be installed immediately.  
0 of these updates are security updates.  
  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
Your Hardware Enablement Stack (HWE) is supported until April 2025.  
Last login: Sat Apr 11 09:00:59 2026 from 192.168.66.2  
[04/11/26]seed@VM:~$
```

然后需要在虚拟机配置代理端口，命令如下，配置后重启 docker。

```
1. # 创建配置目录
2. sudo mkdir -p /etc/systemd/system/docker.service.d
3. # 配置代理
4. sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf <<-'EOF'
5. [Service]
6. Environment="HTTP_PROXY=http://127.0.0.1:1081"
7. Environment="HTTPS_PROXY=http://127.0.0.1:1081"
8. EOF
```

接着打开新的命令行，使用 `ssh -N -R 1081:127.0.0.1:7897 -p 2222 seed@127.0.0.1` 命令建立反向代理，我的代理端口是 7897，此时虚拟机通过 1081 端口连接代理，这样就可以访问国外网站了。该命令运行后无输出即为连接正常。

现在回到 SSH 连接虚拟机终端的命令行，使用 `docker-compose up` 命令发现可以正常启动容器，这样就配置好了基本环境。

```
[04/11/26]seed@VM:~/Labsetup$ docker-compose up
Pulling attacker (handsonsecurity/seed-ubuntu:large)...
large: Pulling from handsonsecurity/seed-ubuntu
da7391352a9b: Pull complete
14428a6d4bcd: Pull complete
2c2d948710f2: Pull complete
b5e99359ad22: Pull complete
3d2251ac1552: Pull complete
1059cf087055: Pull complete
b2afee800091: Pull complete
c2ff2446bab7: Pull complete
4c584b5784bd: Pull complete
Digest: sha256:41efab02008f016a7936d9cadf8e8238146d07c1c12b39cd63c3e73a0297c07a
Status: Downloaded newer image for handsonsecurity/seed-ubuntu:large
Creating seed-attacker ... done
Creating hostA-10.9.0.5 ... done
Creating hostB-10.9.0.6 ... done
Attaching to seed-attacker, hostB-10.9.0.6, hostA-10.9.0.5
hostB-10.9.0.6 | * Starting internet superserver inetd          [ OK ]
hostA-10.9.0.5 | * Starting internet superserver inetd          [ OK ]
```

为了少开几个命令行窗口，我使用 `docker-compose up -d` 让容器在后台运行，这样该命令行可以继续作为终端使用。

3.2 实验任务集 1：使用 Scapy 嗅探和伪造数据包

本次实验我选择第一个任务，先通过 `apt install python3-pip -y` 和 `pip3 install scapy` 安装 scapy 库，准备好实验环境，然后开始正式实验。

3.2.1 嗅探包

在先前的命令行窗口中使用 `docker exec -it seed-attacker /bin/bash` 进入攻击者容器，然后创建 `sniffer.py` 脚本，内容与教程相同，此时的过滤器为 `icmp`。运行该程序后会发现在等待捕获数据包，这时打开另一个命令行窗口，连接虚拟机后使用 `docker exec -it hostA-10.9.0.5 /bin/bash` 命令进入 hostA 容器，使用 `ping 10.9.0.6` 测试和

hostB 的连通性，发现能够正常连通。

```
[04/11/26]seed@VM:~$ docker exec -it hostA-10.9.0.5 /bin/bash
root@9ae193f8c697:/# ping 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data.
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.091 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.299 ms
64 bytes from 10.9.0.6: icmp_seq=3 ttl=64 time=0.103 ms
64 bytes from 10.9.0.6: icmp_seq=4 ttl=64 time=0.104 ms
64 bytes from 10.9.0.6: icmp_seq=5 ttl=64 time=0.128 ms
64 bytes from 10.9.0.6: icmp_seq=6 ttl=64 time=0.093 ms
64 bytes from 10.9.0.6: icmp_seq=7 ttl=64 time=0.104 ms
64 bytes from 10.9.0.6: icmp_seq=8 ttl=64 time=0.279 ms
64 bytes from 10.9.0.6: icmp_seq=9 ttl=64 time=0.103 ms
64 bytes from 10.9.0.6: icmp_seq=10 ttl=64 time=0.075 ms
64 bytes from 10.9.0.6: icmp_seq=11 ttl=64 time=0.098 ms
64 bytes from 10.9.0.6: icmp_seq=12 ttl=64 time=0.133 ms
64 bytes from 10.9.0.6: icmp_seq=13 ttl=64 time=0.103 ms
64 bytes from 10.9.0.6: icmp_seq=14 ttl=64 time=0.081 ms
64 bytes from 10.9.0.6: icmp_seq=15 ttl=64 time=0.116 ms
64 bytes from 10.9.0.6: icmp_seq=16 ttl=64 time=0.147 ms
64 bytes from 10.9.0.6: icmp_seq=17 ttl=64 time=0.255 ms
64 bytes from 10.9.0.6: icmp_seq=18 ttl=64 time=0.088 ms
64 bytes from 10.9.0.6: icmp_seq=19 ttl=64 time=0.116 ms
64 bytes from 10.9.0.6: icmp_seq=20 ttl=64 time=0.080 ms
^C
--- 10.9.0.6 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19426ms
rtt min/avg/max/mdev = 0.075/0.129/0.299/0.064 ms
root@9ae193f8c697:/#
```

在攻击者容器的命令行界面捕获到数据包以及详细信息，如下所示：

```
root@VM:/# root@VM:/# python3 sniffer.py
###[ Ethernet ]###
dst      = 02:42:0a:09:00:06
src      = 02:42:0a:09:00:05
type     = IPv4
###[ IP ]###
version  = 4
ihl      = 5
tos      = 0x0
len      = 84
id       = 48125
flags    = 0F
frag     = 0
ttl      = 64
proto    = icmp
chksum   = 0x6a8f
src      = 10.9.0.5
dst      = 10.9.0.6
options  \
###[ ICMP ]###
type     = echo-request
code     = 0
chksum   = 0xc69c
id       = 0x1b
seq      = 0x1
###[ Raw ]###
load     = '\xdad1\x00\x00\x00\x00\xcf\x07\x00\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&'()*+,-./01234567'

###[ Ethernet ]###
dst      = 02:42:0a:09:00:05
src      = 02:42:0a:09:00:06
type     = IPv4
###[ IP ]###
version  = 4
ihl      = 5
tos      = 0x0
len      = 84
id       = 65494
flags    =
frag     = 0
ttl      = 64
proto    = icmp
chksum   = 0x66b6
src      = 10.9.0.6
dst      = 10.9.0.5
options  \
###[ ICMP ]###
type     = echo-reply
code     = 0
chksum   = 0xc69c
```

说明成功捕获数据包，并通过回调函数打印出具体信息。接下来使用 su seed 切换到普通用户权限，再次运行发现代码报错了，这说明没有 root 权限无法运行嗅探程序。

```

root@VM:/# su seed
seed@VM:/# python3 sniffer.py
Traceback (most recent call last):
  File "sniffer.py", line 6, in <module>
    pkt = sniff(iface='br-028808531ae2', filter='icmp', prn=print_pkt)
  File "/usr/local/lib/python3.8/dist-packages/scapy/sendrecv.py", line 1036, in sniff
    sniffer._run(*args, **kwargs)
  File "/usr/local/lib/python3.8/dist-packages/scapy/sendrecv.py", line 906, in _run
    sniff_sockets[L2socket(type=ETH_P_ALL, iface=iface,
  File "/usr/local/lib/python3.8/dist-packages/scapy/arch/linux.py", line 398, in __init__
    self.ins = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.htons(type)) # noqa: E501
  File "/usr/lib/python3.8/socket.py", line 231, in __init__
    _socket.socket.__init__(self, family, type, proto, fileno)
PermissionError: [Errno 1] Operation not permitted
seed@VM:/#

```

然后验证不同的过滤器的捕获效果，先将过滤器设置为 tcp and src host 10.9.0.5 and dst port 23，这样可以捕获来自特定 IP 且目标端口 23 的 TCP 包。在 hostA 中使用 telnet 10.9.0.6 23 和 hostB 通信，这一过程需要输入账户名和密码。

```

root@9ae193f8c697:/# telnet 10.9.0.6 23
Trying 10.9.0.6...
Connected to 10.9.0.6.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
213d20e8836e login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

seed@213d20e8836e:~$ exit
logout
Connection closed by foreign host.

```

通信时发现攻击者容器捕获到了 TCP 流量，并且将信息拼接可以得到账户名和密码。

```

root@VM:/# python3 sniffer.py
###[ Ethernet ]###
dst      = 02:42:0a:09:00:06
src      = 02:42:0a:09:00:05
type     = IPv4
###[ IP ]###
version  = 4
ihl      = 5
tos      = 0x10
len      = 60
id       = 11071
flags    = DF
frag     = 0
ttl      = 64
proto    = tcp
chksum   = 0xfb50
src      = 10.9.0.5
dst      = 10.9.0.6
\options \
###[ TCP ]###
sport    = 38058
dport    = telnet
seq      = 1535246986
ack      = 0
dataofs  = 10
reserved = 0
flags    = S
window   = 64240
chksum   = 0x144b
urgptr   = 0
options  = [('MSS', 1460), ('SackOK', b''), ('Timestamp', (3725080313, 0)), ('NOP', None), ('WScale', 7)]

###[ Ethernet ]###
dst      = 02:42:0a:09:00:06
src      = 02:42:0a:09:00:05
type     = IPv4
###[ IP ]###
version  = 4
ihl      = 5
tos      = 0x10
len      = 52
id       = 11072
flags    = DF
frag     = 0
ttl      = 64
proto    = tcp
chksum   = 0xfb57
src      = 10.9.0.5
dst      = 10.9.0.6
\options \

```

将过滤器设置为 net 128.230.0.0/16，这时再开启一个命令行并进入攻击者容器，创建 python 文件，将源 IP 设置为 128.230.5.6，目的 IP 设置为 10.9.0.6。发送后在刚刚监听的攻击者命令行界面捕获到相应的数据包。

```

root@VM:/# python3 sniffer.py
###[ Ethernet ]###
dst      = 02:42:0a:09:00:06
src      = 02:42:a9:e1:dc:b8
type     = IPv4
###[ IP ]###
version  = 4
ihl      = 5
tos      = 0x0
len      = 28
id       = 1
flags    = 
frag     = 0
ttl      = 64
proto    = icmp
chksum   = 0xee9
src      = 128.230.1.2
dst      = 10.9.0.6
\options \
###[ ICMP ]###
type     = echo-request
code     = 0
chksum   = 0xf7ff
id       = 0x0
seq      = 0x0

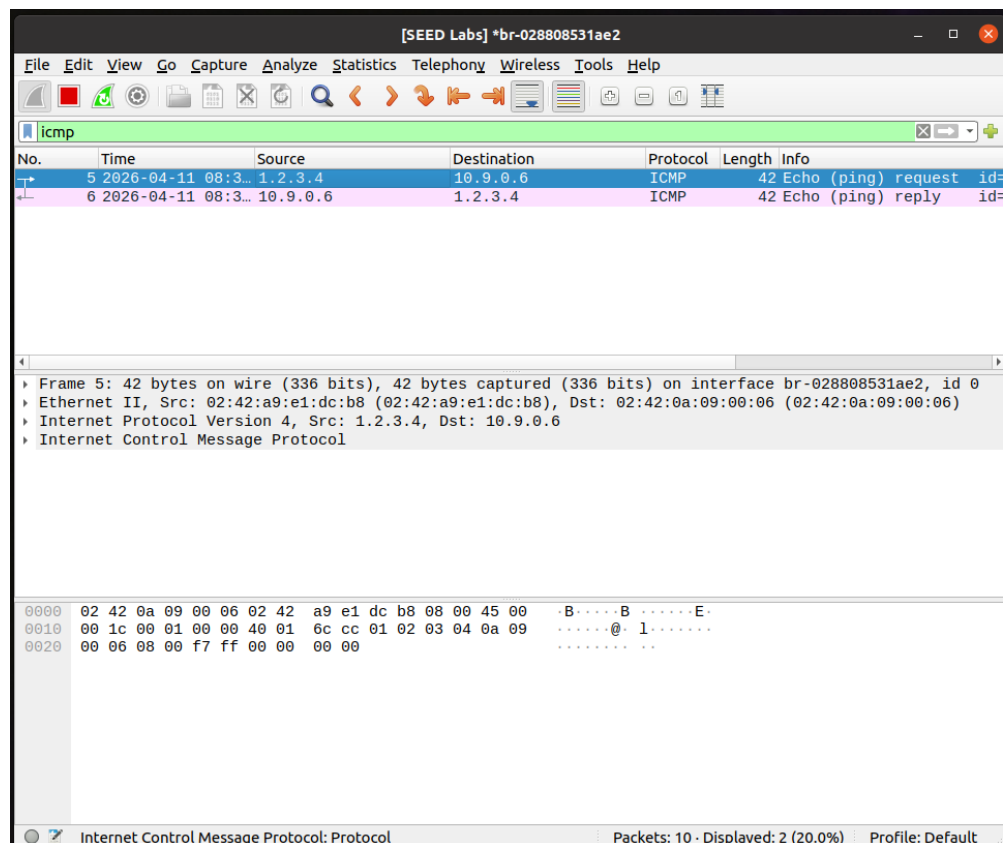
###[ Ethernet ]###
dst      = 02:42:a9:e1:dc:b8
src      = 02:42:0a:09:00:06
type     = IPv4
###[ IP ]###
version  = 4
ihl      = 5
tos      = 0x0
len      = 28
id       = 49258
flags    = 
frag     = 0
ttl      = 64
proto    = icmp
chksum   = 0x2e80
src      = 10.9.0.6
dst      = 128.230.1.2
\options \
###[ ICMP ]###
type     = echo-reply
code     = 0
chksum   = 0xffff
id       = 0x0
seq      = 0x0

```


3.2.2 伪造 ICMP 包

按照教程给出的代码进行实验，创建 icmp_spoof.py，将源 IP 设置为 1.2.3.4，伪造了数据包并发送给 hostB，通过 wireshark 抓包和 tcpdump 监听发现数据包成功发送，并且 hostB 回复给了 1.2.3.4。

```
root@213d20e8836e:/# tcpdump -i eth0 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
12:39:56.001582 IP 1.2.3.4 > 213d20e8836e: ICMP echo request, id 0, seq 0, length 8
12:39:56.001615 IP 213d20e8836e > 1.2.3.4: ICMP echo reply, id 0, seq 0, length 8
```



由于当前的 IP 是伪造的，因此自己不会收到数据包，而接收者会向被伪造 IP 发送回应数据包，这样会导致被伪造 IP 接收很多的数据包，这就是 ICMP 反射攻击。

3.2.3 Traceroute

在这部分内容中我使用代码实现了 TTL 为 1-8 的尝试，目标 IP 为 8.8.8.8。经过尝试发现只有 TTL 为 1 时收到了回应，其他均显示超时。使用 traceroute 之后也发现只有第一跳收到了回应，这说明公网路由器进行了过滤。

```
root@VM:/# python3 test.py
TTL=1: reply from 192.168.66.2
TTL=2 timeout
TTL=3 timeout
TTL=4 timeout
TTL=5 timeout
TTL=6 timeout
TTL=7 timeout
```

```
root@VM:/# traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1  192.168.66.2 (192.168.66.2)  0.159 ms  0.065 ms  0.081 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
30  * * *
```

3.2.4 窃听和伪造结合

这部分基于上个任务的代码，这里收到数据包后复制数据并进行回复，这样不论目标 IP 是什么，hostA 发送的数据包都会收到攻击者伪造的回复。hostA 对不同 IP 测试连通性的结果如下：

这样就完成了本次的数据包嗅探和伪造实验！本次实验让我了解了数据包嗅探和伪造的方法以及 scapy 库的使用方法，在配置环境的时候也学会了端口转发的配置方法。

四、关于人工智能使用的说明

4.1 人工智能使用声明

本人段俊宇承诺：对此文档的所有内容正确性、真实性负责，充分理解了提交报告的知识内容，并对使用大模型辅助的内容进行了正确的标注。

4.2 人工智能使用标注

在本次实验中，环境配置在人工智能和 CSDN 的帮助下完成，其中一些问题由人工智能进行解答。

4.3 人工智能技术使用总结

我使用 deepseek 辅助我完成本次实验，使用过程中，我发现 deepseek 能非常详细地提供实验步骤，以及每一步需要使用的命令，能够较好的帮助我完成实验环境的配置，并解答实验中产生的问题。